

```

1  """Generative pool assignment + score/select helpers for Tier 1 / **540 re-entry**.
```

2

3 =====

4 FOR LATER CHARLES – read this like a notebook CELL map

5 =====

6 Daily path: 540_READ_ME_SIM.md + hero_model_reset_bundle / 540_rho_ablation_bundle.

7 Plain English: 3-Master_Plan/re_entry/04_Pass_A_and_Pass_B_in_Plain_English.md

8

9 -----

10 READ-FIRST GLOSSARY – every symbol below is defined HERE before it is used.

11 Do not skip this block. Later sections assume you already know these words.

12 -----

13

14 WHAT THIS FILE BUILDS

15 A *synthetic* (made-up) basketball-like league: fake players, fake teams,

16 fake “who gets selected.” It is NOT the empirical hero plot. The hero uses

17 real college data; this engine invents data so we can turn knobs (Pass A/B).

18

19 THREE STEPS (always in this order)

20 (1) ASSIGN – put each fake player on exactly one team.

21 (2) SCORE – give each player a ranking number S_i (higher = better for selection).

22 (3) SELECT – pick the winners from those scores (usually the top K).

23

24 Important: SCORE $\neq$ SELECT. Ranking first, then a winner rule.

25

26 CORE LETTERS (people / teams / score)

27 A_i – ability of player i . A made-up talent number we draw from a distribution.

28 T_j – team target for team j . One number per team; soft assignment tries to

29 seat players whose A is near that team’s T .

30 pool – same idea as “team” / roster group. pool_id = which team a player sits on.

31 S_i – selection *score* (ranking weight) for player i . Used only to rank people.

32 K – how many people get selected ($N_{SELECTED}$ in config). “Top K ” = the K

33 highest S_i values under the default winner rule.

34 $Y_{selected}$ – 0/1 outcome after SELECT (1 = got the slot).

35

36 POOL STATISTIC L (what enters the SCORE beside ability)

37 L – a leave-one-out *pool* number for each player: something about their

38 teammates, not about the whole league.

39 LOO – leave-one-out: when computing L for player i , exclude i themselves so

40 their own ability does not inflate their pool measure.

41 L_Q – “quality” L = mean teammate ability (column poolq_loo). Same units as A .

42 This is the sim analog of the hero plot’s X-axis idea (pool quality).

43 L_C – “crowding” L = how packed the roster is with high-ability / “viable”

44 teammates. Lives on $[0, 1]$. Two flavors:

45 hard – share of teammates with $A > \theta$ (column pool_c_loo)

46 smooth – mean of soft viability $\sigma(\gamma(A-\theta))$ (column pool_c_smooth_loo)

47 θ (theta) – viability cutline: ability above θ counts as a “viable peer.”

48 γ (gamma) – sharpness of the soft viability curve (large γ = hard step at θ).

49

50 SCORE FORMULA (Pass A lives here)

51 Talent-only: $S_i = A_i$

52 With pool L : $S_i = A_i - w \cdot L_i$

53 w – weight on the L piece (code name: loo_gap_weight).

54 λ (lambda) – Alex’s name for that same weight in the nesting

55 $S = A - \lambda \cdot L_C$. In this file, $w \approx \lambda$.

56 l_term_scale – NAME DECODE (do not wait for §6):

57 l = pool L (the L above)

58 term = the L -*term* in $S = A - w \cdot L$ (the piece that involves L)

59 scale = multiplier so that L -term is in the same units as A

60 Why needed: L_C is on $[0, 1]$; z-scored A is $\sim -2.5 \dots 3.5$. Without

61 a scale, $w \cdot L_C$ is tiny and congestion barely moves rankings.

62 Config twin name: CROWDING_L_Z_SCALE (same idea).

63

64 ASSIGNMENT KNOBS (Pass B lives here)

65 soft assignment – probabilistic seating: prefer teams with T_j near A_i , but

66 allow overlap (real college talent windows overlap).

67 q (rho) – assortativity strength in soft assignment. 0 = random among

68 open seats; larger q = sharper “sit near your T .”

69 NOT the same as λ/w (those are SCORE weights).

70 σ (sigma) – length-scale in the soft kernel (ability units; default ~ 0.65).

71 τ (tau) – *legacy* temperature knob (old code). Small τ = assortative.

72 Prefer q in 540 work; τ is kept so old notebooks still run.

73 sort-and-chop – sort everyone by ability, cut into equal team slices.

74 Almost no overlap (diagnostic / extreme arm). NOT “ $q \rightarrow \infty$.”

75 preferential attachment – optional rich-get-richer on seats already filling

76 (off for 540 unless you turn the flag on).

```

77
78 SELECT / WINNER RULE
79     choice "C" - deterministic top K by S_i ← default for Pass A and Pass B.
80     choice "A"/"B" - older stochastic rules (kept for 537-era experiments).
81
82 PASS A vs PASS B (what each experiment toggles)
83     Pass A - same assignment; change the SCORE (talent-only vs A - w·L_C).
84     Pass B - same score rule; change ASSIGN only (q / soft vs sort-and-chop).
85
86 OTHER SHORT WORDS YOU WILL SEE
87     z-scored / z-ish - A drawn on a roughly Normal/clipped scale (530-style), not [0,1].
88     unit interval - A drawn on [0,1] (539 beta/uniform). Then L_C needs no scale.
89     coverage / overlap - how much team ability ranges overlap (smoke test: soft » chop).
90     kernel - the soft-match weight formula (gaussian or cauchy) from A to each T.
91
92 SECTION MAP (scroll by banner - terms above already defined)
93     0. Types & column names
94     1. LOO mode helpers (which L column: quality vs crowding)
95     2. AssignmentParams - knobs loaded from tier1_sim_config.py
96     3. Draw abilities A_i and team targets T_j
97     4. ASSIGN - soft_assign / sort-and-chop / build roster table
98     5. LOO pool stats - build L_Q and L_C columns on the roster table
99     6. SCORE - selection_weights (talent-only vs A - w·L); applies l_term_scale
100    7. SELECT - choose_selected / assign_selection
101    8. One-shot league - simulate_generative_rosters (ASSIGN only)
102    9. Smoke test (soft overlap vs sort-and-chop)
103
104 Both Pass A and Pass B call into this file via tier1_generative_eda.run_inverted_u_pipeline.
105
106 Legacy UI: tier1_cell110_playground_run.py (do not use for daily re-entry).
107 Design / overlap forensics: documents/Tier1_Presorting_Design_Note.md
108 Defaults: tier1_sim_config.py
109 =====
110 """
111
112 from __future__ import annotations
113
114 import importlib.util
115 from dataclasses import dataclass
116 from pathlib import Path
117 from typing import Any, Literal
118
119 import numpy as np
120 import pandas as pd
121
122 # =====
123 # 0. TYPES & COLUMN NAMES
124 # =====
125 # These Literals document the allowed strings in config / Pass A-B knobs.
126 # Column constants are the names written onto the players DataFrame after LOO.
127 #
128 # Think of this like 530 CELL 2: every name below is a *setting label*, not math
129 # you have to re-derive. If a string shows up in tier1_sim_config.py or a Pass A/B
130 # arm, it must be one of these allowed values.
131 #
132 # Symbols A_i, T_j, L, LOO, q, λ/w, θ, γ, l_term_scale are defined in the
133 # module docstring READ-FIRST GLOSSARY above - read that before this section.
134
135 # AssignmentKernel: shape of soft-match preference for team target T_j near A_i.
136 # "gaussian" - default; smooth falloff  $\exp(-q (A-T)^2 / (2\sigma^2))$ 
137 # "cauchy" - heavier tails (player still has weight far from T_j)
138 AssignmentKernel = Literal["gaussian", "cauchy"]
139
140 # TargetMeanDist: how we draw each team's fixed target mean T_j (one number per team).
141 # Soft assignment seats players near T_j ≈ A_i; T_j is NOT real college means unless
142 # you choose empirical_530.
143 TargetMeanDist = Literal["uniform", "normal_clipped", "empirical_530"]
144
145 # AbilityDraw: how we invent latent talent A_i for each synthetic player.
146 # If A is z-ish and SCORE later subtracts crowding L (on [0,1]), the score step
147 # multiplies L by l_term_scale (defined in the READ-FIRST GLOSSARY above).
148 AbilityDraw = Literal[
149     "uniform_01", # A ~ U(0,1) - 539-style unit interval
150     "beta_2_2", # A ~ Beta(2,2) on [0,1] - unimodal around 0.5
151     "normal_clipped", # A ~ Normal, clipped - 530-style z-ish (default for 540)
152     "normal_plus_student_t", # normal + Student-t noise - occasional stars/busts
153     "empirical_530", # resample from fitted empirical college perf
154 ]
155
156 # AssignmentMethod: how players land on teams (Pass B toggles this / q).
157 # "soft" - probabilistic match to T_j (overlapping talent windows)
158 # "sort_chop" - sort by ability, chop into equal slices (disjoint; diagnostic)

```

```

159 AssignmentMethod = Literal["soft", "sort_chop"]
160
161 # LooPoolLMode: which leave-one-out pool statistic enters the SCORE as "L".
162 # "quality" - L_Q = mean teammate ability (hero X-axis analog)
163 # "crowding" - L_C hard = share of teammates with A >  $\theta$ 
164 # "crowding_smooth" - L_C soft = mean teammate  $\sigma(\gamma(A-\theta))$ ; Pass A congestion arm
165 LooPoolLMode = Literal["quality", "crowding", "crowding_smooth"]
166
167 # --- Column names written onto the players DataFrame after LOO -----
168 # L_Q - leave-one-out mean teammate ability (hero X-axis analog in sim)
169 POOL_L_QUALITY_COL = "poolq_loo"
170 # L_C hard - LOO share of teammates with ability >  $\theta$  (lives on [0, 1])
171 POOL_L_CROWDING_COL = "pool_c_loo"
172 # L_C smooth - LOO mean of  $\sigma(\gamma(A-\theta))$ ; Pass A congestion arm uses this ([0, 1])
173 POOL_L_CROWDING_SMOOTH_COL = "pool_c_smooth_loo"
174 # Legacy diagnostic - LOO *sum* of teammate ability (not the default score input)
175 POOL_L_CROWDING_SUM_COL = "pool_c_loo_sum"
176
177
178 # =====
179 # 1. LOO MODE HELPERS
180 # =====
181 # Map human / config strings ("quality", "crowding_smooth", ...) → DataFrame columns
182 # and plot labels. Used when building the SCORE (which L enters S_i).
183
184
185 def is_crowding_l_mode(mode: str) -> bool:
186     """True if mode refers to an L_C crowding measure (needs unit scaling vs z-scored A)."""
187     m = str(mode).strip().lower()
188     return m in (
189         "crowding",
190         "l_c",
191         POOL_L_CROWDING_COL,
192         "crowding_smooth",
193         "smooth_crowding",
194         "smooth",
195         POOL_L_CROWDING_SMOOTH_COL,
196     )
197
198
199 def pool_l_column(mode: str) -> str:
200     """Return the players DataFrame column name for the chosen LOO pool L.
201
202     quality          → poolq_loo
203     crowding         → pool_c_loo          (hard viable share)
204     crowding_smooth → pool_c_smooth_loo    (Pass A / Alex smooth L_C)
205     """
206     m = str(mode).strip().lower()
207     if m in ("quality", "l_q", POOL_L_QUALITY_COL):
208         return POOL_L_QUALITY_COL
209     if m in ("crowding_smooth", "smooth_crowding", "smooth", POOL_L_CROWDING_SMOOTH_COL):
210         return POOL_L_CROWDING_SMOOTH_COL
211     if m in ("crowding", "l_c", POOL_L_CROWDING_COL):
212         return POOL_L_CROWDING_COL
213     raise ValueError(
214         f"loo_pool_l_mode must be 'quality', 'crowding', or 'crowding_smooth', got {mode!r}"
215     )
216
217
218 def pool_l_short_label(mode: str) -> str:
219     """Plain-text label for matplotlib axes / sweep logs."""
220     m = str(mode).strip().lower()
221     if m in ("crowding_smooth", "smooth_crowding", "smooth", POOL_L_CROWDING_SMOOTH_COL):
222         return "L_C (smooth viability)"
223     if m in ("crowding", "l_c", POOL_L_CROWDING_COL):
224         return "L_C (viable share)"
225     return "L_Q (LOO mean)"
226
227
228 def pool_l_html_label(mode: str) -> str:
229     """HTML for widgets.HTML - real subscripts via <sub> (legacy CELL 10 UI)."""
230     m = str(mode).strip().lower()
231     if m in ("crowding_smooth", "smooth_crowding", "smooth", POOL_L_CROWDING_SMOOTH_COL):
232         return "L<sub>c</sub><sup>smooth</sup>"
233     if m in ("crowding", "l_c", POOL_L_CROWDING_COL):
234         return "L<sub>c</sub>"
235     return "L<sub>q</sub>"
236
237
238 def pool_l_dropdown_options() -> list[tuple[str, str]]:
239     """(display label, value) for legacy widget dropdowns."""
240     return [

```

```

241         ("Quality - LOO mean", "quality"),
242         ("Crowding - viable share (A > 0)", "crowding"),
243         ("Crowding - smooth viability (539)", "crowding_smooth"),
244     ]
245
246
247 # =====
248 # 2. AssignmentParams - ONE BUNDLE OF KNOBS FOR A LEAGUE DRAW
249 # =====
250 # Mirrors sports/tier1_sim_config.py. Pass A/B bundles load this, then replace()
251 # individual fields (e.g. assignment_rho for Pass B arms).
252
253
254 @dataclass(frozen=True)
255 class AssignmentParams:
256     """Knobs for one generative roster draw (mirror tier1_sim_config.py).
257
258     ASSIGN knobs: n_teams, roster_size, target means, kernel,  $q$ ,  $\sigma$ , preferential.
259     Ability knobs: which distribution for  $A_i$ .
260     Viability knobs:  $\theta$ ,  $\gamma$  for smooth  $L_C$  (used later in SCORE, stored here for convenience).
261
262     Field-by-field (530 CELL 2 style) - what each knob *is*:
263         n_teams / roster_size - league size; total players = product (equal rosters).
264         target_mean_*         - distribution + range for team targets  $T_j$ .
265         assignment_kernel      - gaussian vs cauchy soft-match shape.
266         assignment_temperature- legacy  $\tau$  (small = assortative). Prefer  $q$  below.
267         assignment_rho         - 540 assortativity  $q$  (Pass B knob). 0 = uniform among open.
268         assignment_sigma       - fixed length-scale  $\sigma$  in the soft kernel (default ~0.65).
269         use_preferential_*     - optional rich-get-richer on open seats (off for 540).
270         ability_*              - how  $A_i$  is drawn and clipped.
271         sorting_noise_sd       - only for sort_chop: noise added before the sort.
272         viability_theta        -  $\theta$  outline for "viable peer" ( $L_C$ ); 530 median drafted  $z$ .
273         viability_sharpness    -  $\gamma$  in soft viability  $\sigma(\gamma(A-\theta))$ ; larger = sharper step.
274     """
275
276     # --- League geometry -----
277     n_teams: int
278     # n_teams: how many synthetic teams (pools) in this fake league.
279     roster_size: int
280     # roster_size: how many players sit on every team (fixed equal rosters).
281
282     # --- Team targets  $T_j$  (what soft assignment aims at) -----
283     target_mean_dist: TargetMeanDist
284     # target_mean_dist: which family draws  $T_j$  (uniform / normal_clipped / empirical).
285     target_mean_low: float
286     # target_mean_low: lower clip / uniform low for  $T_j$ .
287     target_mean_high: float
288     # target_mean_high: upper clip / uniform high for  $T_j$ .
289     target_mean_mu: float
290     # target_mean_mu: mean of  $T_j$  when target_mean_dist == "normal_clipped".
291     target_mean_sigma: float
292     # target_mean_sigma: SD of  $T_j$  when target_mean_dist == "normal_clipped".
293
294     # --- Soft assignment kernel (Pass B lives here via  $q$ ) -----
295     assignment_kernel: AssignmentKernel
296     # assignment_kernel: "gaussian" (default) or "cauchy".
297     assignment_temperature: float
298     # assignment_temperature: legacy  $\tau$ . Small  $\tau$  = strong assortativity.
299     # Prefer assignment_rho + assignment_sigma in new / 540 code.
300     assignment_rho: float
301     # assignment_rho: 540 user-facing assortativity  $q$  (Pass B). 0  $\rightarrow$  flat among open seats;
302     # larger  $q$   $\rightarrow$  sharper preference for  $T_j$  near  $A_i$ . NOT the same as  $\lambda$  /  $w$  in SCORE.
303     assignment_sigma: float
304     # assignment_sigma: fixed scale  $\sigma$  in soft kernel (default ~0.65 on ability units).
305     use_preferential_attachment: bool
306     # use_preferential_attachment: if True, multiply kernel by  $(n_j+k)^a$  (rich-get-richer).
307     preferential_alpha: float
308     # preferential_alpha:  $a$  in preferential attachment (ignored when flag is False).
309     preferential_k: float
310     # preferential_k:  $k$  in  $(n_j + k)^a$  - keeps empty teams from going to weight 0.
311
312     # --- Latent ability  $A_i$  -----
313     ability_draw: AbilityDraw
314     # ability_draw: which distribution invents  $A_i$  (see AbilityDraw comments above).
315     ability_mean: float
316     # ability_mean: location for normal_* draws.
317     ability_sd: float
318     # ability_sd: scale for normal_* draws.
319     ability_clip_low: float
320     # ability_clip_low: hard floor after draw (normal_* paths).
321     ability_clip_high: float
322     # ability_clip_high: hard ceiling after draw (normal_* paths).

```

```

323 ability_student_t_df: float
324 # ability_student_t_df: degrees of freedom for Student-t add-on noise.
325 ability_student_t_scale: float
326 # ability_student_t_scale: multiplier on Student-t noise (0 = off).
327
328 # --- Sort-and-chop + viability (L_C) -----
329 sorting_noise_sd: float = 0.0
330 # sorting_noise_sd: ONLY for sort_chop - SD of noise added to ability before sorting.
331 # 0 = pure ability order (537-style hard slices).
332 viability_theta: float = 0.7546158731868137
333 # viability_theta (θ): ability outline for "viable peer." Used when building L_C.
334 # Default ≈ 530 median drafted performance on z scale.
335 viability_sharpness: float = 18.0
336 # viability_sharpness (γ): logistic sharpness in soft viability  $\sigma(\gamma(A-\theta))$ .
337 # Large  $\gamma$  ≈ hard step at  $\theta$ ; small  $\gamma$  = gradual. Used for crowding_smooth.
338
339 @property
340 def n_individuals(self) -> int:
341     """Total synthetic players = teams × roster size (fixed equal rosters)."""
342     return int(self.n_teams) * int(self.roster_size)
343
344 @classmethod
345 def from_module(cls, mod: Any) -> AssignmentParams:
346     """Build params from an already-imported config module object."""
347     return cls(
348         n_teams=int(mod.N_TEAMS),
349         roster_size=int(mod.ROSTER_SIZE),
350         target_mean_dist=mod.TARGET_MEAN_DIST,
351         target_mean_low=float(mod.TARGET_MEAN_LOW),
352         target_mean_high=float(mod.TARGET_MEAN_HIGH),
353         target_mean_mu=float(mod.TARGET_MEAN_MU),
354         target_mean_sigma=float(mod.TARGET_MEAN_SIGMA),
355         assignment_kernel=mod.ASSIGNMENT_KERNEL,
356         assignment_temperature=float(mod.ASSIGNMENT_TEMPERATURE),
357         assignment_rho=float(getattr(mod, "ASSIGNMENT_RHO", 1.0)),
358         assignment_sigma=float(
359             getattr(mod, "ASSIGNMENT_SIGMA", mod.ASSIGNMENT_TEMPERATURE)
360         ),
361         use_preferential_attachment=bool(
362             getattr(mod, "USE_PREFERENTIAL_ATTACHMENT", False)
363         ),
364         preferential_alpha=float(mod.PREFERENTIAL_ALPHA),
365         preferential_k=float(mod.PREFERENTIAL_K),
366         ability_draw=mod.ABILITY_DRAW,
367         ability_mean=float(mod.ABILITY_MEAN),
368         ability_sd=float(mod.ABILITY_SD),
369         ability_clip_low=float(mod.ABILITY_CLIP_LOW),
370         ability_clip_high=float(mod.ABILITY_CLIP_HIGH),
371         ability_student_t_df=float(mod.ABILITY_STUDENT_T_DF),
372         ability_student_t_scale=float(mod.ABILITY_STUDENT_T_SCALE),
373         sorting_noise_sd=float(getattr(mod, "SORTING_NOISE_SD", 0.0)),
374         viability_theta=float(getattr(mod, "VIABILITY_THETA", 0.7546158731868137)),
375         viability_sharpness=float(getattr(mod, "VIABILITY_SHARPNESS", 18.0)),
376     )
377
378 @classmethod
379 def from_tier1_sim_config(cls, path: Path | None = None) -> AssignmentParams:
380     """Load sports/tier1_sim_config.py from disk and build AssignmentParams."""
381     cfg_path = path or Path(__file__).resolve().parent / "tier1_sim_config.py"
382     spec = importlib.util.spec_from_file_location("tier1_sim_config", cfg_path)
383     if spec is None or spec.loader is None:
384         raise ImportError(f"cannot load tier1_sim_config from {cfg_path}")
385     mod = importlib.util.module_from_spec(spec)
386     spec.loader.exec_module(mod)
387     return cls.from_module(mod)
388
389
390 def load_tier1_sim_config(path: Path | None = None) -> Any:
391     """Load sports/tier1_sim_config.py as a module (same pattern as 537 + sim_config).
392
393     Returns the module object so callers can also read selection knobs
394     (N_SELECTED, SELECTION_SCORE_MODE, ...) that live beside AssignmentParams.
395     """
396     cfg_path = path or Path(__file__).resolve().parent / "tier1_sim_config.py"
397     spec = importlib.util.spec_from_file_location("tier1_sim_config", cfg_path)
398     if spec is None or spec.loader is None:
399         raise ImportError(f"cannot load tier1_sim_config from {cfg_path}")
400     mod = importlib.util.module_from_spec(spec)
401     spec.loader.exec_module(mod)
402     return mod
403
404

```



```

405 # =====
406 # 3. DRAW ABILITIES A_i AND TEAM TARGETS T_j
407 # =====
408 # Step 0 of assignment: create the talent deck and the team "targets."
409 # Pass B reuses the SAME A_i / T_j across arms; only placement changes.
410
411
412 def draw_abilities(
413     rng: np.random.Generator,
414     n: int,
415     *,
416     ability_draw: AbilityDraw,
417     ability_mean: float = 0.0,
418     ability_sd: float = 1.0,
419     ability_clip_low: float = -2.5,
420     ability_clip_high: float = 3.5,
421     ability_student_t_df: float = 4.0,
422     ability_student_t_scale: float = 0.25,
423 ) -> np.ndarray:
424     """Draw latent ability A_i for n synthetic players.
425
426     What A_i *is*: a made-up talent number for each fake player. It is NOT taken
427     from the empirical hero panel unless ability_draw == "empirical_530".
428
429     Args (CELL 2 style)
430         rng                - NumPy Generator (caller owns the seed).
431         n                  - how many players to invent (= n_teams × roster_size).
432         ability_draw       - which distribution (see AbilityDraw comments in §0).
433         ability_mean / sd  - location/scale for normal_* draws.
434         ability_clip_*     - hard floor/ceiling after normal_* draws.
435         ability_student_t_* - df + scale for optional heavy-tail add-on.
436
437     Common choices
438         normal_clipped    - z-ish scale (530-style); needs l_term_scale with L_C
439         beta_2_2 / uniform_01 - [0,1] (539-style); L_C needs no extra scale
440         normal_plus_student_t - heavier tails (occasional stars / busts)
441         empirical_530      - resample from fitted empirical perf (optional)
442     """
443     if ability_draw == "uniform_01":
444         return rng.uniform(0.0, 1.0, size=n)
445     if ability_draw == "beta_2_2":
446         # 539 Alex sim: Beta(2,2) on [0,1], unimodal, mean 0.5
447         return rng.beta(2.0, 2.0, size=n)
448     if ability_draw == "normal_clipped":
449         # Clip keeps extreme z draws from blowing up soft kernels / plots.
450         return np.clip(
451             rng.normal(loc=ability_mean, scale=ability_sd, size=n),
452             ability_clip_low,
453             ability_clip_high,
454         )
455     if ability_draw == "normal_plus_student_t":
456         # Base normal, then optional Student-t bumps for fat tails.
457         base = rng.normal(loc=ability_mean, scale=ability_sd, size=n)
458         if ability_student_t_scale > 0 and ability_student_t_df > 0:
459             noise = rng.standard_t(df=ability_student_t_df, size=n)
460             base = base + ability_student_t_scale * noise
461         return np.clip(base, ability_clip_low, ability_clip_high)
462     if ability_draw == "empirical_530":
463         # Optional: pull A_i from the fitted empirical college-perf distribution.
464         from sports_pipeline.empirical_perf_fit import draw_empirical_abilities
465
466         return draw_empirical_abilities(rng, n)
467     raise ValueError(f"unknown ability_draw {ability_draw!r}")
468
469
470 def draw_target_means(
471     rng: np.random.Generator,
472     n_teams: int,
473     *,
474     target_mean_dist: TargetMeanDist,
475     target_mean_low: float,
476     target_mean_high: float,
477     target_mean_mu: float,
478     target_mean_sigma: float,
479 ) -> np.ndarray:
480     """Draw fixed team target means T_j (length n_teams).
481
482     What T_j *is*: one number per team – the “ideal seat” soft assignment aims at.
483     Soft assignment tries to put player i on teams whose T_j ≈ A_i.
484
485     We do NOT copy real college means into T_j for the minimal cross-domain story
486     (Alex); ranges are calibrated so overlap / SD look plausible vs 530 forensics.

```

```

487
488 Args
489     n_teams          - length of the returned T vector.
490     target_mean_dist - uniform / normal_clipped / empirical_530.
491     target_mean_low/high - bounds (uniform range, or clip for normal).
492     target_mean_mu/sigma - normal_clipped location/scale.
493     """
494     if target_mean_dist == "uniform":
495         return rng.uniform(target_mean_low, target_mean_high, size=n_teams)
496     if target_mean_dist == "normal_clipped":
497         raw = rng.normal(loc=target_mean_mu, scale=target_mean_sigma, size=n_teams)
498         return np.clip(raw, target_mean_low, target_mean_high)
499     if target_mean_dist == "empirical_530":
500         from sports_pipeline.empirical_perf_fit import draw_empirical_abilities
501         return draw_empirical_abilities(rng, n_teams)
502     raise ValueError(f"unknown target_mean_dist {target_mean_dist!r}")
503
504
505
506 # =====
507 # 4. ASSIGN - SOFT MATCH (q) OR SORT-AND-CHOP BENCHMARK
508 # =====
509 # This is generative step (1). Pass B varies q / method here.
510 # Soft assignment creates overlapping talent windows (like real 530 CELL 8).
511 # Sort-and-chop creates nearly disjoint slices (coverage ≈ 1) - diagnostic only.
512
513
514 def _kernel_weights(
515     ability_i: float,
516     team_targets: np.ndarray,
517     *,
518     assignment_kernel: AssignmentKernel,
519     assignment_rho: float = 1.0,
520     assignment_sigma: float = 0.65,
521     assignment_temperature: float | None = None,
522 ) -> np.ndarray:
523     """Unnormalized soft-match weights for one player across all teams.
524
525     What these weights *are*: how strongly player i "likes" each team j before
526     we zero out full rosters and turn the vector into probabilities.
527
528     540 default (q parameterization)
529      $w_j \propto \exp(-q \cdot (A_i - T_j)^2 / (2\sigma^2))$  [gaussian]
530      $q = 0 \rightarrow$  all ones (uniform among open rosters)
531      $q \uparrow \rightarrow$  sharper preference for  $T_j$  near  $A_i$ 
532
533     Args
534         ability_i          - this player's  $A_i$  (a scalar).
535         team_targets       -  $T_j$  for every team (length  $n_{teams}$ ).
536         assignment_rho      -  $q$  assortativity (Pass B).
537         assignment_sigma    -  $\sigma$  length-scale in ability units.
538         assignment_temperature - if not None, use legacy  $\tau$  path instead of  $q$ 
539                                (opposite intuition: small  $\tau$  = assortative).
540                                Prefer  $q$  in new / 540 code.
541     """
542     #  $\delta[j] = A_i - T_j$  (how far this player is from each team's target)
543     delta = float(ability_i) - np.asarray(team_targets, dtype=float)
544     if assignment_temperature is not None:
545         # Legacy  $\tau$  path (537-era). Keep for old notebooks; new code should pass  $q$ .
546         tau = max(float(assignment_temperature), 1e-12)
547         if assignment_kernel == "gaussian":
548             return np.exp(-0.5 * (delta / tau) ** 2)
549         if assignment_kernel == "cauchy":
550             return 1.0 / (1.0 + (delta / tau) ** 2)
551         raise ValueError(f"unknown assignment_kernel {assignment_kernel!r}")
552
553     rho = float(assignment_rho)
554     if rho <= 0.0:
555         # Flat weights → after open-roster masking, uniform among open seats.
556         return np.ones(len(team_targets), dtype=float)
557     sigma = max(float(assignment_sigma), 1e-12)
558     z = delta / sigma # distance in units of  $\sigma$ 
559     if assignment_kernel == "gaussian":
560         return np.exp(-0.5 * rho * z**2)
561     if assignment_kernel == "cauchy":
562         return 1.0 / (1.0 + rho * z**2)
563     raise ValueError(f"unknown assignment_kernel {assignment_kernel!r}")
564
565
566 def _normalize_probs(weights: np.ndarray) -> np.ndarray:
567     """Turn non-negative weights into a probability vector (sums to 1)."""
568     w = np.asarray(weights, dtype=float)

```

```

569 w = np.where(np.isfinite(w), w, 0.0)
570 w = np.clip(w, 0.0, None)
571 total = float(w.sum())
572 if total <= 0:
573     raise ValueError("non-positive total assignment weight")
574 return w / total
575
576
577 def soft_assign(
578     rng: np.random.Generator,
579     ability: np.ndarray,
580     team_targets: np.ndarray,
581     roster_size: int,
582     *,
583     assignment_kernel: AssignmentKernel = "gaussian",
584     assignment_rho: float = 1.0,
585     assignment_sigma: float = 0.65,
586     assignment_temperature: float | None = None,
587     preferential_alpha: float = 0.0,
588     preferential_k: float = 1.0,
589 ) -> np.ndarray:
590     """Place each player on exactly one team with equal roster sizes (soft  $\pi_{ij}$ ).
591
592     What this returns
593     pool_id[i] = integer team index for player i (length = n_players)
594
595     Algorithm (sequential fill – like dealing into piles with capacity)
596     1. Shuffle player order (so seat-filling order is random, not ability-sorted).
597     2. For each player i, compute kernel weights toward each team target  $T_j$ .
598     3. Optional preferential attachment: multiply by  $(n_j + k)^a$ 
599        (default  $a=0$  / off for 540 – leave this alone unless you turn the flag on).
600     4. Zero out teams already at roster_size (full = cannot sit there).
601     5. Sample one team from the remaining probabilities; increment that team's count.
602
603     Args (CELL 2 style)
604     ability          –  $A_i$  vector already drawn.
605     team_targets     –  $T_j$  vector already drawn (length n_teams).
606     roster_size      – seats per team; len(ability) must equal n_teams * roster_size.
607     assignment_rho    –  $q$  assortativity (Pass B). 0 = uniform among open seats.
608     assignment_sigma  –  $\sigma$  length-scale in the kernel (ability units).
609     assignment_temperature – if set, use legacy  $\tau$  path instead of  $q$  (prefer  $q$ ).
610     preferential_alpha/k – rich-get-richer;  $a=0$  means off.
611
612     Default kernel (540):  $\exp(-q (A_i - T_j)^2 / (2\sigma^2))$ .
613     """
614     ability = np.asarray(ability, dtype=float)
615     team_targets = np.asarray(team_targets, dtype=float)
616     n_players = len(ability)
617     n_teams = len(team_targets)
618     expected = n_teams * int(roster_size)
619     if n_players != expected:
620         raise ValueError(
621             f"len(ability)={n_players} must equal n_teams*roster_size={expected}"
622         )
623
624     # pool_id: which team each player lands on (-1 until assigned).
625     pool_id = np.full(n_players, -1, dtype=np.int64)
626     # counts[j]: how many players already seated on team j (must end = roster_size).
627     counts = np.zeros(n_teams, dtype=np.int64)
628     alpha = float(preferential_alpha)
629     k_pref = max(float(preferential_k), 1e-12)
630
631     for i in rng.permutation(n_players):
632         # Unnormalized soft-match weights for player i across all teams.
633         w = _kernel_weights(
634             ability[i],
635             team_targets,
636             assignment_kernel=assignment_kernel,
637             assignment_rho=assignment_rho,
638             assignment_sigma=assignment_sigma,
639             assignment_temperature=assignment_temperature,
640         )
641         if alpha != 0.0:
642             # Preferential attachment: teams that already have more players get
643             # extra weight (optional; off when  $a=0$ ).
644             w = w * np.power(counts.astype(float) + k_pref, alpha)
645         # open_mask: True where the team still has an empty seat.
646         open_mask = counts < int(roster_size)
647         w = np.where(open_mask, w, 0.0)
648         if float(w.sum()) <= 0.0:
649             # Tiny  $\tau$  or extreme  $A_i$  vs  $T_j$  can zero out the kernel; place uniformly
650             # among teams that still have roster room.

```



```

651         open_idx = np.flatnonzero(open_mask)
652         if open_idx.size == 0:
653             raise RuntimeError(
654                 "soft_assign: no open roster slots "
655                 f"(counts={counts}, roster_size={roster_size})"
656             )
657         w = np.zeros(n_teams, dtype=float)
658         w[open_idx] = 1.0
659         pool_id[i] = int(rng.choice(n_teams, p=_normalize_probs(w)))
660         counts[pool_id[i]] += 1
661
662     if np.any(counts != int(roster_size)):
663         raise RuntimeError(f"uneven rosters after soft_assign: {counts}")
664     return pool_id
665
666
667 def assign_sort_chop_benchmark(
668     rng: np.random.Generator,
669     ability: np.ndarray,
670     n_teams: int,
671     *,
672     sorting_noise_sd: float = 0.0,
673 ) -> np.ndarray:
674     """537-style hard assortative assignment: sort by ability, equal-count slices.
675
676     What this *is*: Pass B "sort_chop" arm / 530 CELL 8 red-line benchmark.
677     Sort everyone by ability (optionally + noise), then chop into n_teams contiguous
678     blocks of equal size. Talent windows barely overlap (coverage ≈ 1).
679
680     Important: this is NOT " $q \rightarrow \infty$ ". Soft  $q$  and sort-chop are different mechanisms.
681     """
682     ability = np.asarray(ability, dtype=float)
683     n = len(ability)
684     noise_sd = max(float(sorting_noise_sd), 0.0)
685     # signal: what we sort on. Pure ability when noise_sd=0; else ability + noise.
686     signal = (
687         ability
688         if noise_sd == 0.0
689         else ability + rng.normal(0.0, noise_sd, size=n)
690     )
691     order = np.argsort(signal, kind="mergesort")
692     # base: team labels in ability order – team 0 gets the lowest block, etc.
693     base = np.repeat(np.arange(n_teams), int(np.ceil(n / n_teams)))[:n]
694     pool_id = np.empty(n, dtype=np.int64)
695     pool_id[order] = base
696     return pool_id
697
698
699 def build_roster_dataframe(
700     ability: np.ndarray,
701     pool_id: np.ndarray,
702     team_targets: np.ndarray,
703 ) -> pd.DataFrame:
704     """One row per player after assignment – the table SCORE/SELECT will enrich.
705
706     Columns written here (before LOO / selection)
707     player_id    – 0 ... n-1
708     ability      –  $A_i$  (latent talent)
709     pool_id      – which team this player sits on
710     team_target  –  $T_j$  for that player's team (looked up from team_targets[pool_id])
711
712     Later steps add: poolq_loo / pool_c_* (LOO), selection_weight, Y_selected.
713     """
714     ability = np.asarray(ability, dtype=float)
715     pool_id = np.asarray(pool_id, dtype=np.int64)
716     team_targets = np.asarray(team_targets, dtype=float)
717     return pd.DataFrame(
718         {
719             "player_id": np.arange(len(ability), dtype=np.int64),
720             "ability": ability,
721             "pool_id": pool_id,
722             "team_target": team_targets[pool_id],
723         }
724     )
725
726
727 # =====
728 # 5. LOO POOL STATS –  $L_Q$  AND  $L_C$  ON EACH FAKE ROSTER
729 # =====
730 # After assignment, each player sees teammates. Leave-one-out (exclude self):
731 #  $L_Q$  = mean teammate ability
732 #  $L_C$  = congestion among viable peers (hard share or smooth viability)

```

```

733 # Pass A congestion arm uses crowding_smooth (pool_c_smooth_loo).
734
735
736 def _default_viability_theta() -> float:
737     """Fallback  $\theta$  if not passed – usually 530 median drafted perf on z scale."""
738     try:
739         return float(
740             AssignmentParams.from_tier1_sim_config().viability_theta
741         )
742     except Exception:
743         return 0.7546158731868137
744
745
746 def _viability_logistic(ability: pd.Series, *, theta: float, gamma: float) -> pd.Series:
747     """Soft viability  $\sigma(\gamma(A-\theta))$  elementwise; NaN where ability is NaN.
748
749      $\theta$  = outline,  $\gamma$  = sharpness. This is crowding softness – NOT assignment  $q$ 
750     and NOT selection noise.
751     """
752     a = pd.to_numeric(ability, errors="coerce")
753     z = gamma * (a - theta)
754     z = z.clip(-500.0, 500.0)
755     return (1.0 / (1.0 + np.exp(-z))).where(a.notna(), np.nan)
756
757
758 def add_loo_pool_columns(
759     players: pd.DataFrame,
760     *,
761     viability_theta: float | None = None,
762     viability_sharpness: float = 18.0,
763 ) -> pd.DataFrame:
764     """Add  $L_Q$  (poolq_loo) and  $L_C$  leave-one-out teammate stats to ``players``.
765
766     Leave-one-out (LOO) means: for player  $i$  on a team, compute the pool statistic
767     from *everyone else on that team* – exclude  $i$  – so your own ability does not
768     inflate your pool quality / crowding.
769
770     Requires columns already present: ability, pool_id.
771
772     Writes (what each column *is*)
773     poolq_loo          -  $L_Q$ : mean teammate ability (hero X-axis analog)
774     pool_c_loo         -  $L_C$  hard: share of teammates with ability  $> \theta \in [0,1]$ 
775     pool_c_smooth_loo  -  $L_C$  soft: mean teammate  $\sigma(\gamma(A-\theta)) \in [0,1]$ 
776     pool_c_loo_sum     - legacy: sum of teammate ability (diagnostic only)
777
778     Args
779     viability_theta ( $\theta$ ) - outline for “viable peer”; None → config default.
780     viability_sharpness ( $\gamma$ ) - logistic sharpness for smooth  $L_C$ .
781     """
782     theta = (
783         float(viability_theta)
784         if viability_theta is not None
785         else _default_viability_theta()
786     )
787     gamma = float(viability_sharpness)
788     out = players.copy()
789
790     # --- Team totals (include self); we subtract self next for LOO -----
791     g = out.groupby("pool_id", observed=True)["ability"]
792     ssum = g.transform("sum") # sum of ability on i's team (incl. i)
793     cnt = g.transform("count").astype(float) # roster size on i's team
794     den = (cnt - 1.0).replace(0.0, np.nan) # LOO denominator = teammates only
795     own = pd.to_numeric(out["ability"], errors="coerce") #  $A_i$  for this row
796     loo_sum = ssum - own # sum of teammate ability (exclude self)
797     out[POOL_L_CROWDING_SUM_COL] = loo_sum
798     #  $L_Q$  = mean of teammates' ability (exclude self)
799     out[POOL_L_QUALITY_COL] = loo_sum / den
800
801     # --- Hard crowding  $L_C$ : LOO share of teammates above  $\theta$  -----
802     # above: 1.0 if this player's  $A > \theta$ , else 0.0 (then we LOO that indicator).
803     above = (own > theta).astype(float)
804     out["_above_theta"] = above
805     sum_above = out.groupby("pool_id", observed=True)["_above_theta"].transform("sum")
806     own_above = out["_above_theta"].where(own.notna(), np.nan)
807     # loo_count: how many *other* teammates sit above  $\theta$ 
808     loo_count = sum_above - own_above.fillna(0.0)
809     loo_count = loo_count.where(own.notna(), np.nan)
810     # LOO pool size = roster count - 1 (exclude self), same denominator as  $L_Q$ 
811     loo_pool_n = (cnt - 1.0).replace(0.0, np.nan)
812     loo_share = loo_count / loo_pool_n #  $\in [0,1]$  when defined
813     loo_share = loo_share.where(own.notna(), np.nan)
814     loo_share = loo_share.where(cnt >= 2.0, np.nan) # need  $\geq 1$  teammate

```

```

815 out[POOL_L_CROWDING_COL] = loo_share
816
817 # --- Smooth crowding L_C: LOO mean of soft viability  $\sigma(\gamma(A-\theta))$  -----
818 viability = _viability_logistic(out["ability"], theta=theta, gamma=gamma)
819 out["_viability"] = viability
820 sum_v = out.groupby("pool_id", observed=True)[ "_viability" ].transform("sum")
821 own_v = viability
822 loo_smooth = (sum_v - own_v) / den # mean teammate soft-viability  $\in [0,1]$ 
823 loo_smooth = loo_smooth.where(own.notna(), np.nan)
824 loo_smooth = loo_smooth.where(cnt >= 2.0, np.nan)
825 out[POOL_L_CROWDING_SMOOTH_COL] = loo_smooth
826 out = out.drop(columns=["_above_theta", "_viability"])
827 return out
828
829
830 def add_poolq_loo(players: pd.DataFrame) -> pd.DataFrame:
831     """Backward-compatible alias: adds both L_Q and L_C columns."""
832     return add_loo_pool_columns(players)
833
834
835 # =====
836 # 6. SCORE - TURN (A, L) INTO A RANKING WEIGHT S_i
837 # =====
838 # Generative step (2). Pass A toggles here:
839 #   ability          → S = A (talent-only /  $\lambda = 0$ )
840 #   loo_gap_plus_ability → S = (1-w)A + w(A-L) = A - w·L
841 #   with L = crowding_smooth → congestion-in-score story
842 #
843 # NAME DECODE + UNIT MATCHING - parameter l_term_scale
844 # -----
845 # Full decode is in the module docstring READ-FIRST GLOSSARY (top of file).
846 # Short reminder only here (you should already know this before §6):
847 #
848 #   l = pool L; term = L-piece of S = A - w·L; scale = unit-matching multiplier.
849 #   L used in score = L_C × l_term_scale
850 #   None → auto ≈ (p90 of A - p10 of A). Skip scaling if A already on [0,1].
851 #   Config twin: CROWDING_L_Z_SCALE.
852
853
854 def ability_on_unit_interval(ability: np.ndarray) -> bool:
855     """True when synthetic A_i looks drawn on [0, 1] (539 beta/uniform), not z-scored.
856
857     Used only to decide whether crowding L needs l_term_scale.
858     Heuristic: min ≥ -0.05 and max ≤ 1.05 after dropping non-finite values.
859     """
860     finite = np.asarray(ability, dtype=float)
861     finite = finite[np.isfinite(finite)]
862     if finite.size == 0:
863         return False
864     return float(finite.min()) >= -0.05 and float(finite.max()) <= 1.05
865
866
867 def default_crowding_l_z_scale(ability: np.ndarray) -> float:
868     """Auto value for l_term_scale when A is z-scored and L is crowding on [0,1].
869
870     What it *is*: p90(A) - p10(A) - a robust "typical spread" of ability.
871     Fallback 4.0 if too few finite values or a near-zero spread.
872
873     Why: map L_C  $\in [0,1]$  into ability units so w·L_C can actually move S_i.
874     Otherwise congestion is numerically invisible next to z-scored A.
875     """
876     finite = np.asarray(ability, dtype=float)
877     finite = finite[np.isfinite(finite)]
878     if finite.size < 10:
879         return 4.0
880     spread = float(np.nanpercentile(finite, 90) - np.nanpercentile(finite, 10))
881     return spread if spread > 1e-6 else 4.0
882
883
884 def effective_l_for_selection(
885     l_values: np.ndarray,
886     ability: np.ndarray,
887     *,
888     pool_l_mode: str,
889     l_term_scale: float | None = None,
890 ) -> np.ndarray:
891     """Return L in the same units as ability for subtractive scores.
892
893     Args
894     l_values - raw LOO L from the players table (L_Q or L_C column).
895     ability - A_i vector (used only to decide / auto-pick the scale).
896     pool_l_mode - "quality" / "crowding" / "crowding_smooth".

```

```

897     l_term_scale = OPTIONAL. Name decode: l = pool L; term = the L-piece of
898     S = A - w·L; scale = unit-matching multiplier.
899     What the *number* is: converts L_C ∈ [0,1] into ability
900     units: L_used = L_C × l_term_scale. Full story: §6 banner.
901     None → auto = default_crowding_l_z_scale(ability)
902     (= p90(A)-p10(A)). Ignored for quality mode and for
903     unit-interval ability draws.
904
905 Behavior
906     L_Q (quality)                → pass through (already ability units).
907     L_C + A on [0,1]             → pass through (already comparable).
908     L_C + A z-scored             → multiply by l_term_scale (or auto).
909     """
910     l = np.asarray(l_values, dtype=float)
911     # Quality L is already in ability units – no scaling.
912     if not is_crowding_l_mode(pool_l_mode):
913         return l
914     # Unit-interval A (beta/uniform): L_C ∈ [0,1] is already comparable – no scale.
915     if ability_on_unit_interval(ability):
916         return l
917     # Z-scored A + crowding L: pick / apply l_term_scale.
918     scale = (
919         float(l_term_scale)
920         if l_term_scale is not None and np.isfinite(l_term_scale) and l_term_scale > 0
921         else default_crowding_l_z_scale(ability)
922     )
923     # L_used_in_score = L_C × scale (now in ability units)
924     return l * scale
925
926 def selection_weights(
927     players: pd.DataFrame,
928     *,
929     score_mode: str,
930     loo_gap_weight: float,
931     pool_l_mode: str = "quality",
932     l_term_scale: float | None = None,
933 ) -> np.ndarray:
934     """Compute score / ranking weight for each player (the S_i vector).
935
936     What S_i *is*: a number used only to *rank* players for selection. Higher
937     weight → more likely (or certain, under choice C) to get Y_selected=1.
938     This is the SCORE step; SELECT happens in choose_selected.
939
940     score_mode
941     "ability"
942         S_i = A_i                                     ← Pass A talent-only arm (λ = 0)
943     "loo_gap_plus_ability"
944         S_i = w·(A_i - L_i) + (1-w)·A_i
945             = A_i - w·L_i                             ← Pass A congestion arm when L = L_C
946             loo_gap_weight is w (Alex λ story in code).
947
948     Args (CELL 2 style)
949     score_mode      - "ability" or "loo_gap_plus_ability" (see above).
950     loo_gap_weight  - w in A - w·L. 0 ≈ talent-only; 1 ≈ full gap A-L.
951                     Same role as Alex λ in the nesting S = A - λ·L_C.
952     pool_l_mode     - which L column: quality / crowding / crowding_smooth.
953     l_term_scale    - name: l = pool L; term = L-piece of S=A-w·L; scale =
954                     unit matcher. Number: L_used = L_C × this. See §6.
955                     None = auto. Only when crowding mode AND A not on [0,1].
956     """
957     # a: raw ability A_i
958     a = players["ability"].to_numpy(dtype=float)
959     # lcol: which DataFrame column holds the chosen L (poolq_loo / pool_c_*)
960     lcol = pool_l_column(pool_l_mode)
961     # q_raw: L as stored on the table (L_C still on [0,1] here if crowding)
962     q_raw = players[lcol].to_numpy(dtype=float)
963     # q: L after unit matching (may be L_C × l_term_scale)
964     q = effective_l_for_selection(
965         q_raw, a, pool_l_mode=pool_l_mode, l_term_scale=l_term_scale
966     )
967     mode = str(score_mode).strip().lower()
968     if mode == "ability":
969         # Talent-only: ignore L entirely.
970         w = a.copy()
971     elif mode == "loo_gap_plus_ability":
972         # Congestion-in-score: S = A - w·L (algebraically same as the mix form).
973         wgt = float(loo_gap_weight) # w / λ in the nesting
974         w = wgt * (a - q) + (1.0 - wgt) * a
975         # If L was missing for a row, fall back to talent-only for that row.
976         w = np.where(np.isfinite(q_raw), w, a)
977     else:

```

```

979         raise ValueError(f"unknown selection score_mode {score_mode!r}")
980     w = np.where(np.isfinite(w), w, 0.0)
981     # Clip at 0: ranking weights are non-negative for stochastic choices A/B too.
982     return np.clip(w, 0.0, None)
983
984
985 # =====
986 # 7. SELECT - WINNER RULE GIVEN SCORES
987 # =====
988 # Generative step (3). v1 / Pass A-B default: choice "C" = top K by score.
989 # Choices A/B are stochastic draws (kept for older 537 experiments).
990
991
992 def choose_selected(
993     rng: np.random.Generator,
994     weights: np.ndarray,
995     k: int,
996     choice: str,
997 ) -> np.ndarray:
998     """Boolean mask of length n: True = selected (drafted / advanced).
999
1000     What this *is*: the SELECT step. Takes the score vector S_i (= weights) and
1001     picks exactly who gets the slot. Score ≠ select - ranking first, then this.
1002
1003     choice (winner rule)
1004     "A" - sample K players without replacement α weights (stochastic)
1005     "B" - independent Bernoulli with p α weight (stochastic; expected ~K)
1006     "C" - deterministic top K by weight ← Pass A/B default (winner rule)
1007
1008     Args
1009     weights - S_i from selection_weights (non-negative ranking scores).
1010     k        - how many players get selected (N_SELECTED in config).
1011     choice   - "A" / "B" / "C" as above.
1012     """
1013     n = len(weights)
1014     w = np.asarray(weights, dtype=float)
1015     w = np.where(np.isfinite(w), w, 0.0)
1016     w = np.clip(w, 0.0, None)
1017     # n_positive: how many players have a usable positive score (can be picked).
1018     n_positive = int(np.count_nonzero(w > 0))
1019     if n_positive == 0:
1020         return np.zeros(n, dtype=bool)
1021     # k_eff: cannot select more people than have positive weight.
1022     k_eff = min(int(k), n_positive)
1023     if choice == "A":
1024         # Stochastic proportional sample without replacement.
1025         total = float(w.sum())
1026         if total <= 0:
1027             p = np.full(n, 1.0 / n)
1028         else:
1029             p = w / total
1030         idx = rng.choice(n, size=k_eff, replace=False, p=p)
1031         out = np.zeros(n, dtype=bool)
1032         out[idx] = True
1033         return out
1034     if choice == "B":
1035         # Independent coin flips; expected number selected ≈ k_eff.
1036         total = float(w.sum())
1037         p = w / total if total > 0 else np.full(n, 1.0 / n)
1038         return rng.uniform(size=n) < np.minimum(p * k_eff, 1.0)
1039     if choice == "C":
1040         # Deterministic: the K highest scores win (ties broken by mergesort stability).
1041         idx = np.argsort(w, kind="mergesort")[-k_eff:]
1042         out = np.zeros(n, dtype=bool)
1043         out[idx] = True
1044         return out
1045     raise ValueError(f"unknown winner_selection {choice!r}")
1046
1047
1048 def resolve_crowding_l_z_scale(
1049     crowding_l_z_scale: float | None,
1050     *,
1051     pool_l_mode: str,
1052 ) -> float | None:
1053     """Decide the l_term_scale value that selection_weights should use.
1054
1055     What this returns
1056     A positive float → use that as l_term_scale (multiply L_C by it).
1057     None             → tell effective_l_for_selection to AUTO-pick
1058                       (p90(A)-p10(A)), or skip scaling if not needed.
1059
1060     Priority

```



```

1061     1. Explicit crowding_l_z_scale argument if it is a finite positive number.
1062     2. Else, if pool_l_mode is a crowding mode, read CROWDING_L_Z_SCALE from
1063         tier1_sim_config.py when that constant is set and positive.
1064     3. Else None (auto inside effective_l_for_selection, or no-op for quality).
1065
1066     Only relevant for crowding L modes. Quality mode always gets None here.
1067     Same idea as l_term_scale – two names for one knob (API vs config).
1068     """
1069     if crowding_l_z_scale is not None:
1070         val = float(crowding_l_z_scale)
1071         if np.isfinite(val) and val > 0:
1072             return val
1073         return None
1074     if not is_crowding_l_mode(pool_l_mode):
1075         return None
1076     try:
1077         mod = load_tier1_sim_config()
1078         # CROWDING_L_Z_SCALE in config = same concept as l_term_scale in the API.
1079         raw = getattr(mod, "CROWDING_L_Z_SCALE", None)
1080         if raw is not None:
1081             val = float(raw)
1082             if np.isfinite(val) and val > 0:
1083                 return val
1084     except (ImportError, OSError, TypeError, ValueError):
1085         pass
1086     return None
1087
1088
1089 def assign_selection(
1090     players: pd.DataFrame,
1091     rng: np.random.Generator,
1092     *,
1093     n_selected: int,
1094     score_mode: str,
1095     loo_gap_weight: float,
1096     winner_selection: str,
1097     pool_l_mode: str = "quality",
1098     viability_theta: float | None = None,
1099     viability_sharpness: float | None = None,
1100     crowding_l_z_scale: float | None = None,
1101 ) -> pd.DataFrame:
1102     """SCORE then SELECT on an already-assigned roster table.
1103
1104     Pipeline inside this function (after ASSIGN already happened)
1105     1. add_loo_pool_columns → attach L_Q / L_C to each row
1106     2. selection_weights → S_i (ranking score)
1107     3. choose_selected → Y_selected ∈ {0,1}
1108
1109     Domain-agnostic name: "selected" can mean draft / tenure / promotion.
1110
1111     Args (CELL 2 style)
1112     players          – roster table with ability + pool_id (from ASSIGN).
1113     n_selected       – K: how many get Y_selected=1 (top-K under choice C).
1114     score_mode       – "ability" or "loo_gap_plus_ability" (Pass A toggle).
1115     loo_gap_weight    – w / λ in S = A - w·L (Pass A congestion strength).
1116     winner_selection – "A"/"B"/"C"; Pass A/B default "C" = top K by score.
1117     pool_l_mode      – which L enters the score (quality / crowding_smooth...).
1118     viability_theta   – θ for L_C; None → config default.
1119     viability_sharpness – γ for smooth L_C; None → config default.
1120     crowding_l_z_scale – explicit l_term_scale for crowding L (None → config/auto).
1121                        See $6 banner: converts L_C ∈ [0,1] into ability units.
1122
1123     Writes onto the returned copy
1124     poolq_loo / pool_c_* – LOO pool stats
1125     selection_weight     – S_i for each player
1126     Y_selected           – 1 if selected, else 0
1127     """
1128     gamma = (
1129         float(viability_sharpness)
1130         if viability_sharpness is not None
1131         else float(AssignmentParams.from_tier1_sim_config().viability_sharpness)
1132     )
1133     # Step 1 – attach LOO L columns (L_Q and both L_C flavors).
1134     out = add_loo_pool_columns(
1135         players,
1136         viability_theta=viability_theta,
1137         viability_sharpness=gamma,
1138     )
1139     # l_scale: resolved l_term_scale (explicit arg → config → None=auto).
1140     l_scale = resolve_crowding_l_z_scale(
1141         crowding_l_z_scale, pool_l_mode=pool_l_mode
1142     )

```

```

1143 # Step 2 - SCORE: S_i for every player (may use L_C × l_scale).
1144 w = selection_weights(
1145     out,
1146     score_mode=score_mode,
1147     loo_gap_weight=loo_gap_weight,
1148     pool_l_mode=pool_l_mode,
1149     l_term_scale=l_scale,
1150 )
1151 # Step 3 - SELECT: winner rule turns scores into 0/1 outcomes.
1152 out["Y_selected"] = choose_selected(
1153     rng, w, int(n_selected), str(winner_selection)
1154 ).astype(int)
1155 out["selection_weight"] = w
1156 return out
1157
1158 # Legacy names (early 538 drafts used "promotion" instead of "selection")
1159 promotion_weights = selection_weights
1160 choose_promoted = choose_selected
1161
1162
1163
1164 def assign_promotion(
1165     players: pd.DataFrame,
1166     rng: np.random.Generator,
1167     *,
1168     n_promoted: int,
1169     score_mode: str,
1170     loo_gap_weight: float,
1171     winner_selection: str,
1172     pool_l_mode: str = "quality",
1173     viability_theta: float | None = None,
1174     viability_sharpness: float | None = None,
1175     crowding_l_z_scale: float | None = None,
1176 ) -> pd.DataFrame:
1177     """Legacy wrapper: same as assign_selection, also writes Y_promoted."""
1178     out = assign_selection(
1179         players,
1180         rng,
1181         n_selected=n_promoted,
1182         score_mode=score_mode,
1183         loo_gap_weight=loo_gap_weight,
1184         winner_selection=winner_selection,
1185         pool_l_mode=pool_l_mode,
1186         viability_theta=viability_theta,
1187         viability_sharpness=viability_sharpness,
1188         crowding_l_z_scale=crowding_l_z_scale,
1189     )
1190     out["Y_promoted"] = out["Y_selected"]
1191     out["promotion_weight"] = out["selection_weight"]
1192     return out
1193
1194
1195 def roster_team_stats(df: pd.DataFrame) -> pd.DataFrame:
1196     """Per-pool realized mean, SD, min, max of ability (530 CELL 5-8 style).
1197
1198     Used for Plot A overlap diagnostics: soft should have high coverage_peak;
1199     sort-and-chop should sit near coverage = 1.
1200     """
1201     g = df.groupby("pool_id", sort=True)["ability"]
1202     out = g.agg(["mean", "std", "min", "max", "count"]).rename(
1203         columns={"mean": "pool_mean", "std": "pool_sd", "count": "roster_n"}
1204     )
1205     out["team_target"] = df.groupby("pool_id", sort=True)["team_target"].first()
1206     return out.reset_index()
1207
1208
1209 # =====
1210 # 8. ONE-SHOT LEAGUE DRAW - ASSIGN ONLY (score/select happen upstream)
1211 # =====
1212 # simulate_generative_rosters = draw A, draw T, assign → players + team summary.
1213 # Pass A/B then call assign_selection (via tier1_generative_eda) on that table.
1214
1215
1216 def simulate_generative_rosters(
1217     params: AssignmentParams | None = None,
1218     *,
1219     rng: np.random.Generator | None = None,
1220     seed: int | None = 42,
1221     method: AssignmentMethod = "soft",
1222     ability: np.ndarray | None = None,
1223     team_targets: np.ndarray | None = None,
1224 ) -> tuple[pd.DataFrame, pd.DataFrame, np.ndarray]:

```

```

1225 """One full synthetic league **assignment** draw (ASSIGN only).
1226
1227 What this does NOT do: score or select. Pass A/B call assign_selection
1228 (via tier1_generative_eda) afterward on the players table returned here.
1229
1230 Optional ability / team_targets: Pass B passes the SAME draws into every arm
1231 so only the assignment method / q changes (fair comparison across q).
1232
1233 Args (CELL 2 style)
1234     params          - AssignmentParams bundle; None → load tier1_sim_config.py.
1235     rng / seed      - RNG; if rng is None, build one from seed (default 42).
1236     method          - "soft" (q kernel) or "sort_chop" (hard slices).
1237     ability         - optional pre-drawn A_i (Pass B reuse). None → draw now.
1238     team_targets    - optional pre-drawn T_j (Pass B reuse). None → draw now.
1239
1240 Returns
1241 -----
1242 players : DataFrame
1243     player_id, ability, pool_id, team_target (no Y_selected yet)
1244 teams : DataFrame
1245     per-pool summary (pool_mean, pool_sd, min, max, team_target, ...)
1246 team_targets : ndarray
1247     length n_teams - the T_j used (returned so Pass B can reuse them)
1248 """
1249 if params is None:
1250     params = AssignmentParams.from_tier1_sim_config()
1251 if rng is None:
1252     rng = np.random.default_rng(seed)
1253
1254 n = params.n_individuals # n_teams × roster_size
1255 if ability is None:
1256     # Invent latent talent A_i for every synthetic player.
1257     ability = draw_abilities(
1258         rng,
1259         n,
1260         ability_draw=params.ability_draw,
1261         ability_mean=params.ability_mean,
1262         ability_sd=params.ability_sd,
1263         ability_clip_low=params.ability_clip_low,
1264         ability_clip_high=params.ability_clip_high,
1265         ability_student_t_df=params.ability_student_t_df,
1266         ability_student_t_scale=params.ability_student_t_scale,
1267     )
1268 else:
1269     ability = np.asarray(ability, dtype=float)
1270     if len(ability) != n:
1271         raise ValueError(f"len(ability)={len(ability)} != n_individuals={n}")
1272 if team_targets is None:
1273     # Invent one target mean T_j per team (soft-match aims here).
1274     team_targets = draw_target_means(
1275         rng,
1276         params.n_teams,
1277         target_mean_dist=params.target_mean_dist,
1278         target_mean_low=params.target_mean_low,
1279         target_mean_high=params.target_mean_high,
1280         target_mean_mu=params.target_mean_mu,
1281         target_mean_sigma=params.target_mean_sigma,
1282     )
1283 else:
1284     team_targets = np.asarray(team_targets, dtype=float)
1285     if len(team_targets) != params.n_teams:
1286         raise ValueError(
1287             f"len(team_targets)={len(team_targets)} != n_teams={params.n_teams}"
1288         )
1289
1290 # Preferential attachment off unless config boolean is True (a=0 → no effect).
1291 pref_alpha = (
1292     float(params.preferential_alpha)
1293     if params.use_preferential_attachment
1294     else 0.0
1295 )
1296
1297 if method == "soft":
1298     # Probabilistic seating near T_j; q from params.assignment_rho (Pass B).
1299     pool_id = soft_assign(
1300         rng,
1301         ability,
1302         team_targets,
1303         params.roster_size,
1304         assignment_kernel=params.assignment_kernel,
1305         assignment_rho=params.assignment_rho,
1306         assignment_sigma=params.assignment_sigma,

```

```

1307         preferential_alpha=pref_alpha,
1308         preferential_k=params.preferential_k,
1309     )
1310     elif method == "sort_chop":
1311         # Hard sorted slices - diagnostic / Pass B extreme arm (not  $q \rightarrow \infty$ ).
1312         pool_id = assign_sort_chop_benchmark(
1313             rng,
1314             ability,
1315             params.n_teams,
1316             sorting_noise_sd=params.sorting_noise_sd,
1317         )
1318     else:
1319         raise ValueError(f"unknown method {method!r}")
1320
1321     # players: one row per person; teams: one row per pool for overlap diagnostics.
1322     players = build_roster_dataframe(ability, pool_id, team_targets)
1323     teams = roster_team_stats(players)
1324     return players, teams, team_targets
1325
1326
1327 # =====
1328 # 9. SMOKE TEST - soft overlap should beat sort-and-chop
1329 # =====
1330
1331
1332 def _smoke_compare_overlap() -> None:
1333     """Quick stdout check: soft assignment should raise interval overlap vs sort-chop.
1334
1335     Run: python sports/tier1_pool_assignment.py
1336     Expect: soft coverage_peak » 1; sort_chop coverage_peak near 1;
1337            soft median_pool_sd closer to ~0.8 z than chop.
1338     """
1339     params = AssignmentParams.from_tier1_sim_config()
1340     rng = np.random.default_rng(42)
1341
1342     def coverage_peak(teams: pd.DataFrame, grid: np.ndarray) -> float:
1343         lo = teams["min"].to_numpy()
1344         hi = teams["max"].to_numpy()
1345         cov = np.zeros(len(grid), dtype=float)
1346         for a, b in zip(lo, hi):
1347             cov += (grid >= a) & (grid <= b)
1348         return float(cov.max())
1349
1350     grid = np.linspace(-2.0, 2.0, 81)
1351     _, teams_soft, _ = simulate_generative_rosters(params, rng=rng, method="soft")
1352     _, teams_chop, _ = simulate_generative_rosters(params, rng=rng, method="sort_chop")
1353     peak_soft = coverage_peak(teams_soft, grid)
1354     peak_chop = coverage_peak(teams_chop, grid)
1355     med_sd_soft = float(teams_soft["pool_sd"].median())
1356     med_sd_chop = float(teams_chop["pool_sd"].median())
1357     print(
1358         f"soft: coverage_peak={peak_soft:.0f} median_pool_sd={med_sd_soft:.3f} | "
1359         f"sort_chop: coverage_peak={peak_chop:.0f} median_pool_sd={med_sd_chop:.3f}"
1360     )
1361
1362
1363 if __name__ == "__main__":
1364     _smoke_compare_overlap()

```